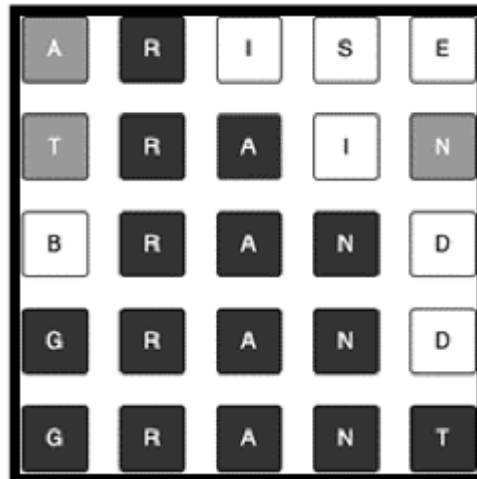


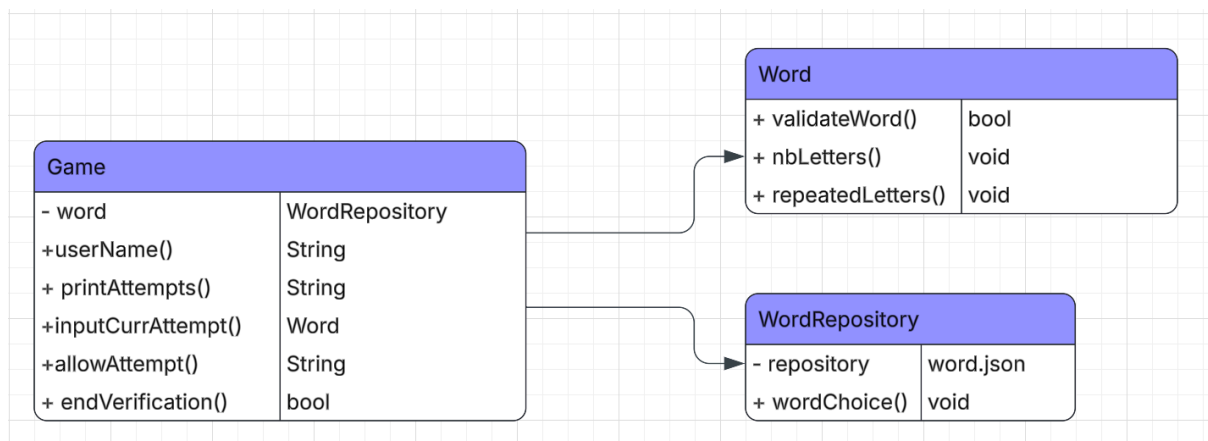
SAE S2.01 : Programmer le jeu Find My Word



Ce projet fait référence au célèbre jeu "Wordle", où le but est de trouver un mot 6 essais maximum. Nous avons dû le programmer en Java, et voici un rapport des étapes et des décisions techniques auxquelles nous avons fait face.

Explication des choix réalisés et décisions techniques :

Lors de cette SAE, nous devions tout d'abord faire une architecture. Nous avons tout d'abord pensé à faire cette UML en premier choix :



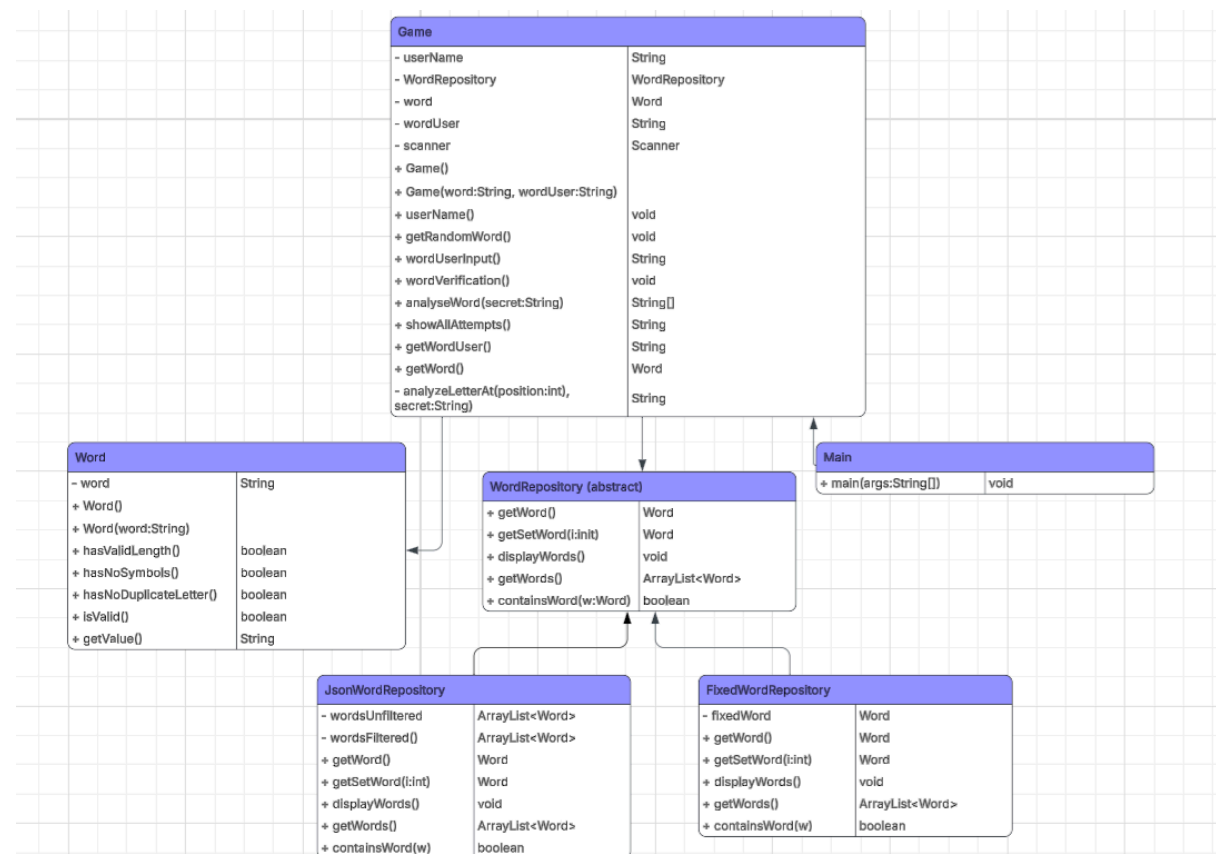
Ce choix était dû à plusieurs facteurs. Tout d'abord, nous devions pouvoir faire le lien entre Game et WordRepository afin d'accéder aux mots et en générer un au hasard. Celui-ci aurait été stocké dans word, qui aurait contenu un des mots (qu'on aurait eu grâce au wordChoice qui fait appel au word.json).

Une fois ce choix fait, nous aurions pu démarrer une partie. Le joueur aurait choisi un mot et grâce à celui-ci, nous aurions vérifié la validité de celui-ci grâce à la classe Word,

Sauf que plusieurs problèmes étaient liés à cela. Tout d'abord, le WordRepository n'a que le word.json et le choix du mot qui apparaît, nous n'avons aucune manière de récupérer les mots. Ensuite, il n'y a pas vraiment d'héritage, donc on ne peut pas changer la source des mots, ce qui est moins extensible.

De plus, on ne sépare pas clairement les responsabilités : le choix du mot ne devrait pas se faire dans WordRepository mais plutôt dans le Game.

Nous devons ainsi modifier notre UML afin d'avoir une meilleure structure. L'UML final ressemble donc à cela :



Les choix sont différents. En effet, dans la première version, les responsabilités n'étaient pas bien séparées. Ici, chaque classe est spécialisée : Game gère seulement la logique du jeu : on centralise donc cette logique afin de simplifier le code. Il y a la classe Word s'occupe des règles et de la validation du mot et c'est une classe autonome. Elle fait en sorte de valider les mots et de faire en sorte que chaque objet de sa classe respecte des règles, comme sur la longueur des mots ou des lettres en double. Et WordRepository gère l'accès aux mots.

Nous avons aussi changé la structure de WordRepository, qui est devenue une classe abstraite. Le fait de faire cela nous a permis de pouvoir obtenir un mot, vérifier si le mot existe et de récupérer la liste des mots. Ainsi, nous ne dépendons pas d'une seule source de mots, mais de plusieurs. Toujours par rapport aux sources, un autre choix se portait sur la mise en place de l'héritage. Nous avons séparé deux implémentations : JsonWordRepository et FixedWordRepository. Nous pouvons donc changer la source de mots sans avoir à modifier la classe Game, et standardiser les comportements puisque ces classes utilisent la classe abstraite de WordRepository.

Aussi, la classe Word était auparavant une classe qui contenait surtout des fonctions. Mais à présent, elle a aussi des objets, des attributs comme word ou wordUser. Cet attribut est un objet lui-même car il se valide grâce à sa classe. Ainsi, les mots sont encapsulés. Ainsi, on empêche les modifications depuis l'extérieur de la classe. Les accès se font uniquement grâce aux méthodes getValue() et getWord(). Cela garantit la sécurité des données et leur cohérence.

Puis, la mise en place de l'héritage est importante. Grâce à celui-ci, on peut spécialiser un comportement. Sans cela, la classe Game dépendait d'une classe de gestion des mots. Mais grâce à l'héritage, Game ne dépend que de WordRepository et "ne connaît pas" l'origine des mots, c'est-à-dire qu'on peut changer la source de mots sans modifier Game directement.

Par rapport au polymorphisme, il apparaît avec WordRepository, FixedWordRepository et JsonWordRepository. La classe mère, une classe abstraite, est WordRepository, tandis que les classes filles sont JsonWordRepository et FixedWordRepository. Il y a bien du polymorphisme car il y a une redéfinition des méthodes : chacune à les classes getWord(), getWords() et containsWord(). Grâce à celui-ci, une variable qui est de type WordRepository peut aussi bien référencer un JsonWordRepository qu'un FixedWordRepository. Java sélectionne la bonne implémentation de la méthode lors de l'exécution en fonction de l'objet. Ainsi, le code est plus flexible.

Maintenant, parlons de la classe Game en particulier, qui gère la logique du jeu. Cette classe analyse le mot lettre par lettre et non en entier. Ce choix a été fait car la consigne était qu'il n'y allait pas avoir de lettres dupliquées. Ainsi, si une lettre apparaît en première position, nous savons à l'avance qu'elle n'allait pas être présente dans la suivante. Cela permet avec certitude qu'un doublon de "présent" pour une même lettre ne serait pas possible puisqu'il n'existe pas de tel mot dans notre projet.

Cet algorithme est basé grâce au `charAt()` , et `indexOf()` qui permettent de retrouver la position de la lettre.

Les choix techniques sont donc basés afin de rendre le projet plus gérable et utilisable : nous avons une bonne gestion du jeu, des mots et du stockage des mots séparés.

Difficultés rencontrées :

Une première difficulté était de bien comprendre et modéliser la structure du Java. En effet, nous devions faire l'UML et c'était la partie qui nous a pris le plus de temps. Nous avons pensé et changé plusieurs fois notre structure, se demandant quelle classe devrait hériter de laquelle, qui devait gérer quoi.

Finalement, après avoir modélisé une structure stable, nous avons commencé à programmer notre projet. Une autre difficulté durant celui-ci est qu'il fallait bien structurer et planifier nos tâches, puisque ce projet et ce jeu avaient plusieurs mécaniques, il fallait faire petit à petit chacune d'entre elles. Par exemple, s'occuper de la vérification d'un mot, puis ensuite, gérer la logique du jeu.